

Patrón CQRS

Command Query Responsibility Segregation

Separación Command Query

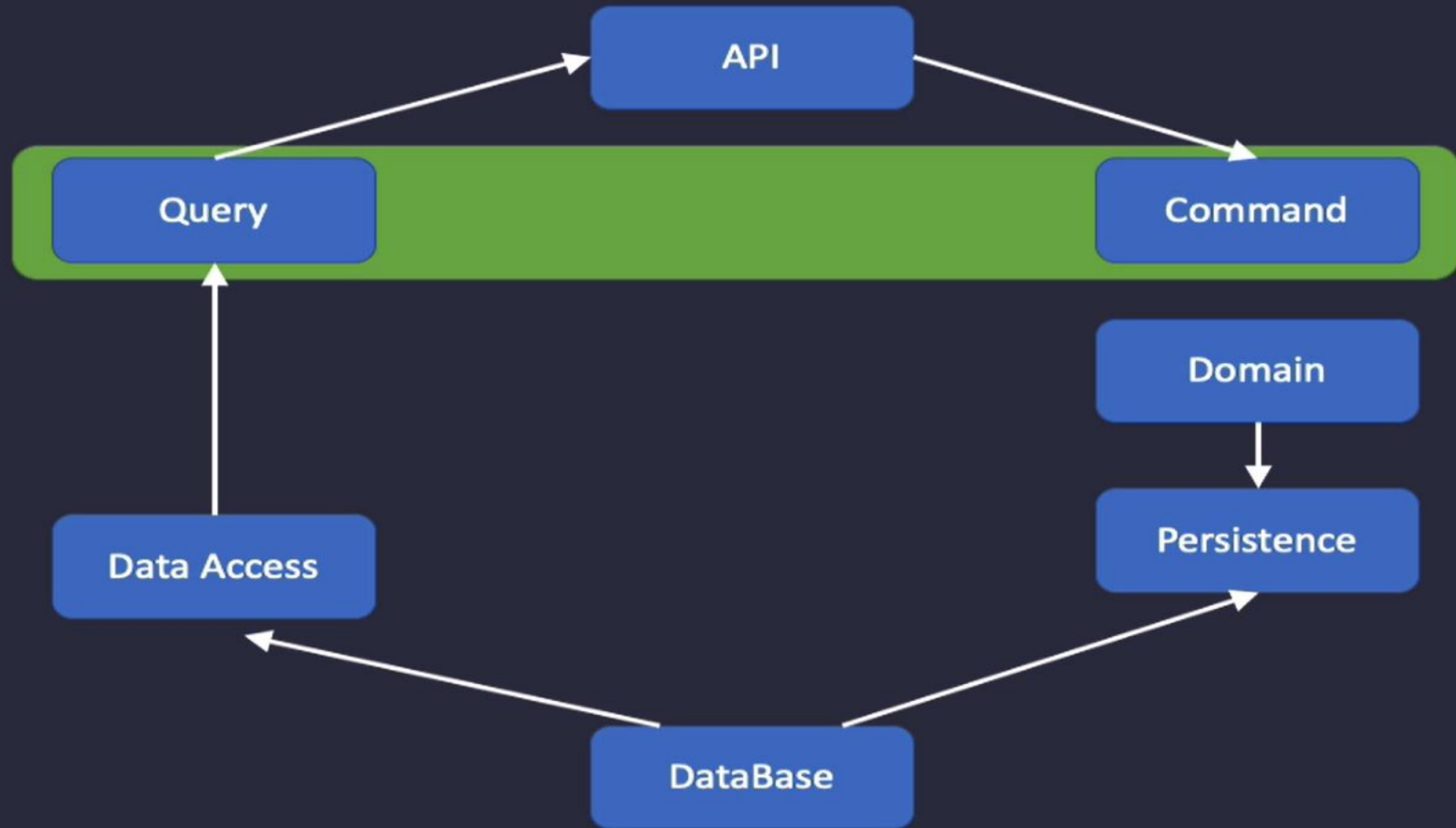
Command

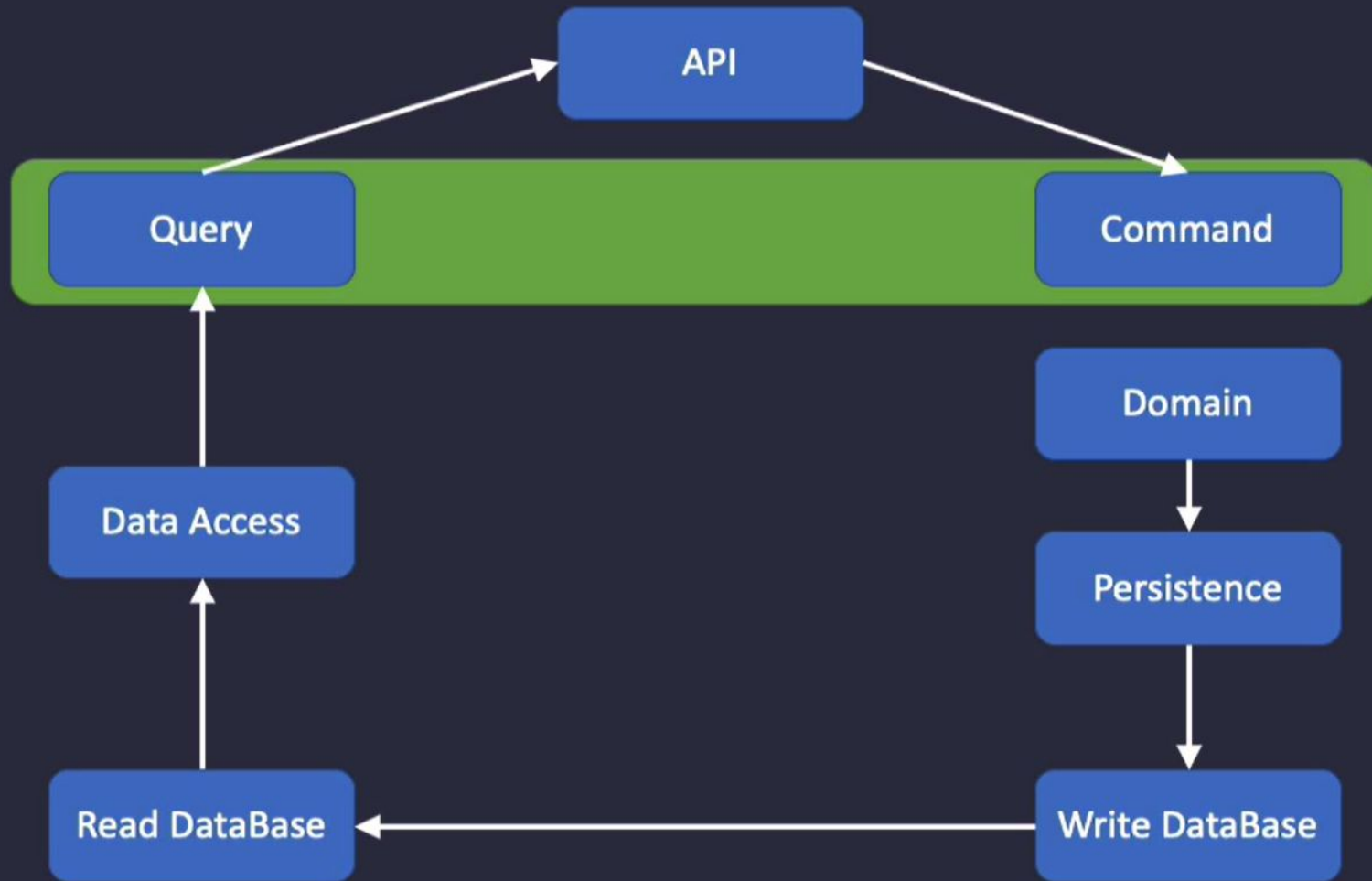
- Realiza una petición
- Modifica el State
- No retorna un valor

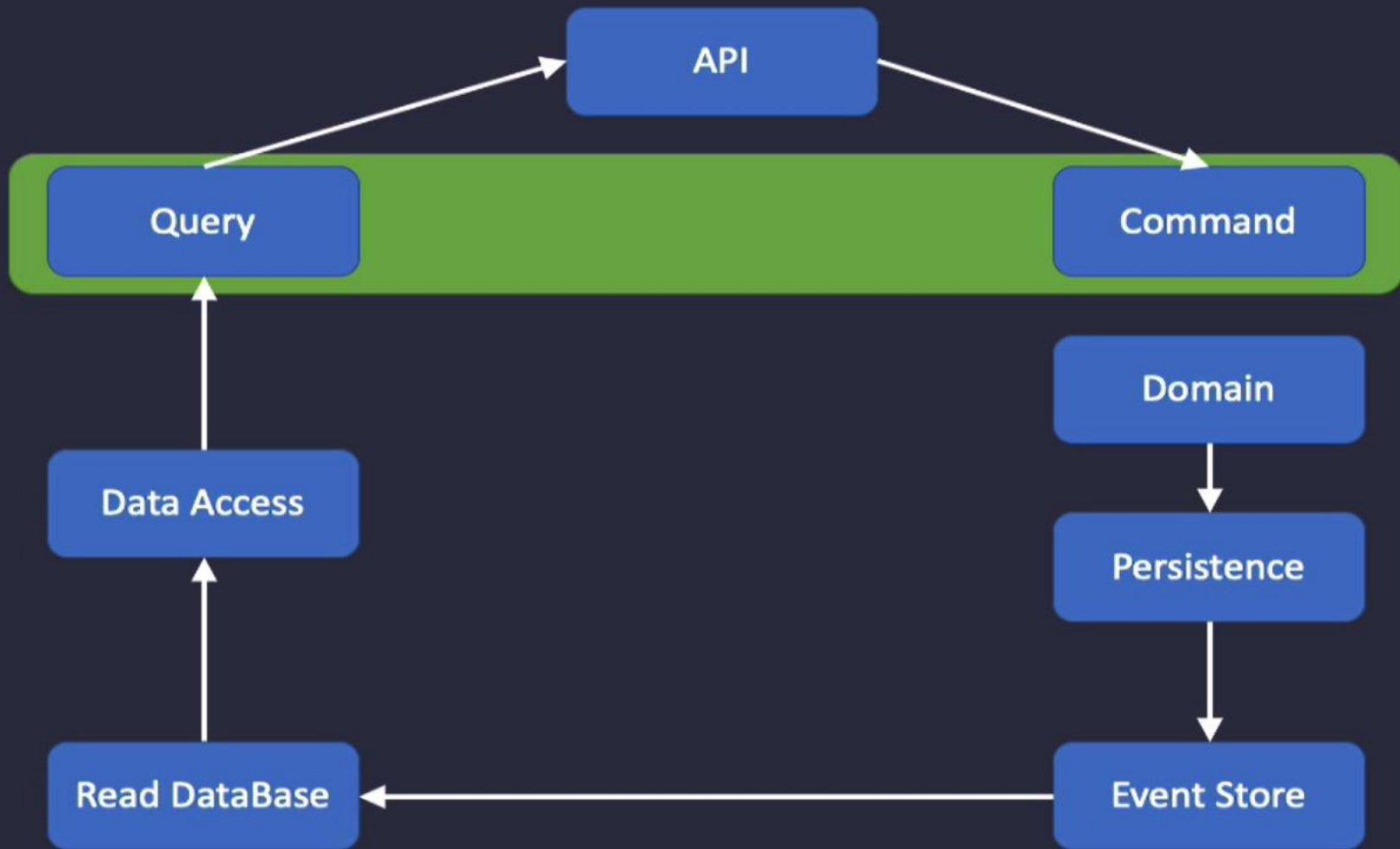


Query

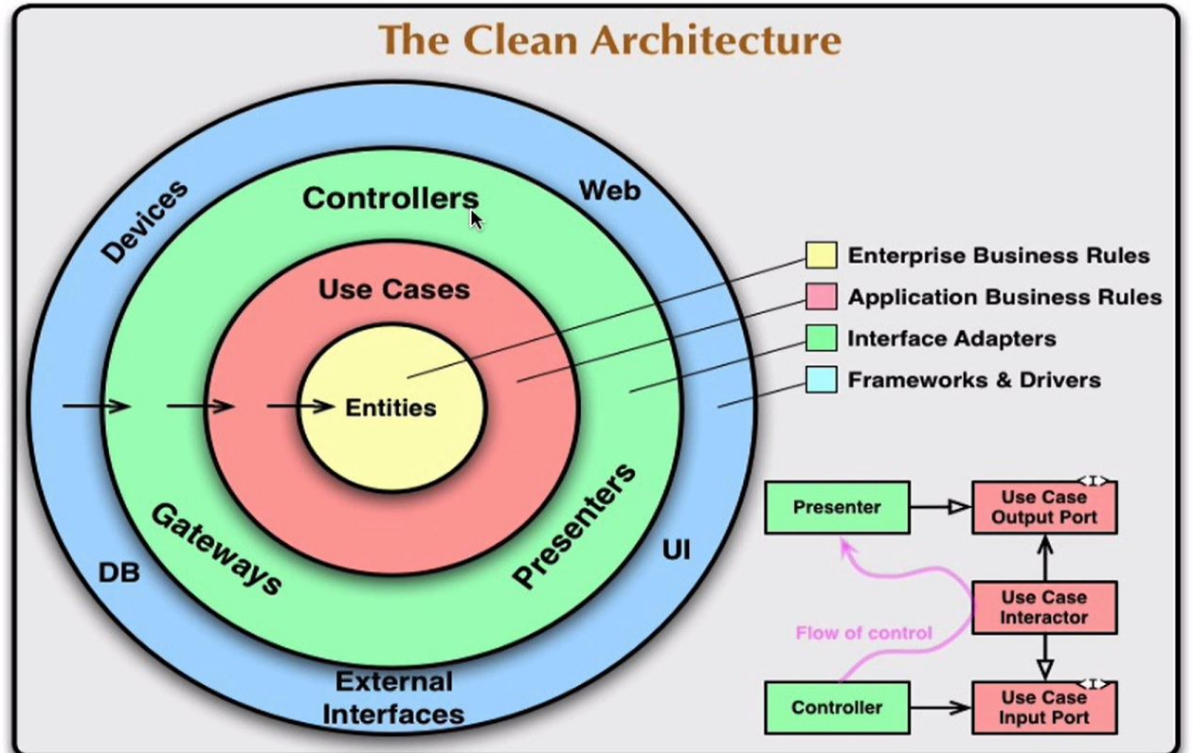
- Realiza una pregunta
- No modifica el state
- Debe retornar un valor

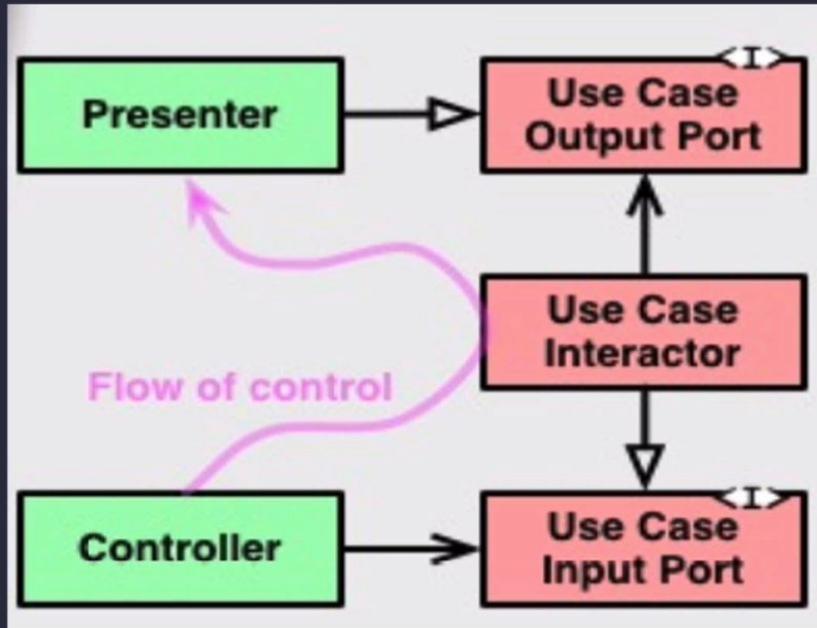






MediaTR



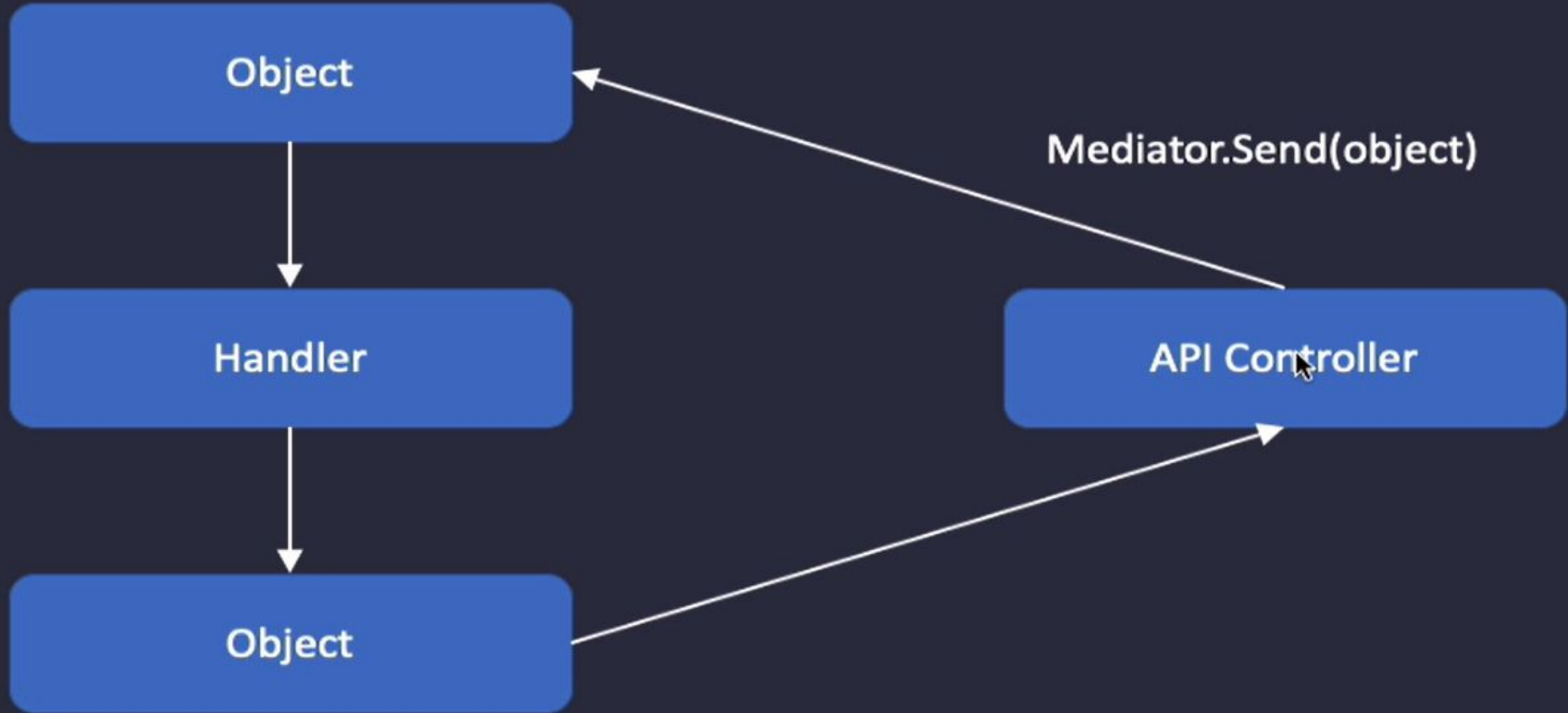


Object Out

Handler

Object In





Implementar MediaTR

1- Instalar MediaTR package. llamado MediaTR.Extensions.DependencyInjection

2- Crear Handler con clases anidadas dentro del proyecto Application, por ejemplo:

```
using MediaTR;
using Domain; //Cliente
using System.Collections.Generic; //List
using Persistence; //DataContext
using Microsoft.EntityFrameworkCore;

public class ListaDeCliente{
    public class Query : IRequest<List<Cliente>>
    {
    }

    public class Handler : IRequestHandler<Query, List<Cliente>> // recibe query/command y el response que en este caso es una lista de cliente
    {
        private DataContext _context;
        public Handler() //inyectar el data context en el constructor
        {
            _context = context ?? throw new ArgumentNullException(nameof(context));
        }
        public async Task<List<Cliente>> Handle(Query request, CancellationToken cancellationToken) //metodo que implementa interfaz IRequestHandler
        {
            //Throw new System.NotImplementedException();
            return await _context.Cliente.ToListAsync();
        }
    }
}
```

Implementar MediaTR

3- Agregar MediaTR como servicio en [program.cs/startup.cs](#) agregar: `services.AddMediaTR(typeof(ListaDeCliente.Handler).Assembly);` //solo basta con agregar el primer handler para que el framework sepa donde se encuentran los demás.

4- Agregar el MediaTR en el controlador, inyectar en el constructor y usarlo:

```
private IMediaTR _mediator

public ClienteController(IMediator mediator){

    _mediator = mediator ?? throw new ArgumentNullException(nameof(mediator))

}

[HttpGet]

public async Task<ActionResult<List<ListaDeCliente>>> Listar(){

    return await _mediator.Send(new ListaDeCliente.Query()); //en mediaTR siempre enviamos eventos por eso usamos el metodo Send()

}
```

Unit Of Work

- El Patrón Unit of Work es un patrón de diseño que se utiliza para agrupar varias operaciones (usualmente de una base de datos) en una sola "unidad de trabajo", asegurando que todas estas operaciones se ejecuten o no se ejecuten.

```
public interface IWorkingExperienceRepository
{
    Task Insert(List<Workingexperience> workingExperiences);
}

public class WorkingExperienceRepository : IWorkingExperienceRepository
{
    private readonly CursoEfContext _context;

    public WorkingExperienceRepository(CursoEfContext cursoEfContext)
    {
        _context = cursoEfContext;
    }

    public async Task Insert(List<Workingexperience> workingExperiences)
        => await _context.Wokringexperiences.AddRangeAsync(workingExperiences);
}
```

```
public interface IUnitOfWork : IDisposable
{
    Task<int> Save();
}

public class UnitOfWork : IUnitOfWork
{
    private readonly CursoEfContext _context;

    public UnitOfWork(CursoEfContext context)
    {
        _context = context;
    }

    public async Task<int> Save()
        => await _context.SaveChangesAsync();

    public void Dispose()
    {
        _context.Dispose();
    }
}
```

```
public interface IUnitOfWork : IDisposable
{
    IUserRepository UserRepository { get; }
    IWorkingExperienceRepository WorkingExperienceRepository { get; }
    Task<int> Save();
}

public class UnitOfWork : IUnitOfWork
{
    public IUserRepository UserRepository { get; }
    public IWorkingExperienceRepository WorkingExperienceRepository { get; }
    private readonly CursoEfContext _context;

    public UnitOfWork(CursoEfContext context, IUserRepository userRepository,
        IWorkingExperienceRepository workingExperienceRepository)
    {
        _context = context;
        UserRepository = userRepository;
        WorkingExperienceRepository = workingExperienceRepository;
    }

    public async Task<int> Save()
        => await _context.SaveChangesAsync();

    public void Dispose()
    {
        _context.Dispose();
    }
}
```

```
builder.Services.AddScoped<IUserRepository, UserRepository>();  
builder.Services.AddScoped<IWorkingExperienceRepository, WorkingExperienceRepository>();  
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
```

```

//Código original
public class RelationsController : Controller
{
    private readonly CursoEfContext _context;

    public RelationsController(CursoEfContext context)
    {
        _context = context;
    }

    [HttpPost("InsertDataExample1")]
    public async Task InsertDataExample1()
    {
        User user1 = new User()
        {
            Email = $"{Guid.NewGuid()}@mail.com",
            UserName = "id1"
        };

        List<Workingexperience> workingExperiences1 = new List<Workingexperience>()
        {
            new Workingexperience()
            {
                UserId = user1.Id,
                Name = "experience 1",
                Details = "details1",
                Environment = "environment"
            },
            new Workingexperience()
            {
                UserId = user1.Id,
                Name = "experience 2",
                Details = "details2",
                Environment = "environment"
            }
        };

        await _context.Users.AddAsync(user1);
        await _context.Wokringexperiences.AddRangeAsync(workingExperiences1);
        await _context.SaveChangesAsync();
    }
}

```